



دليل فهم ومناقشة مشروع دوائي

من المشكلة إلى المعمارية، ومن السيناريوهات إلى الذكاء الاصطناعي

نسخة مهيأة للمناقشة الجامعية
قراءة مركزة • أمثلة عملية • أسئلة متوقعة
أيلول 2026

كيف تستخدم هذا الدليل؟

الهدف ليس حفظ الشاشات؛ الهدف أن تفهم لماذا بني النظام بهذه الطريقة وكيف تتدفق البيانات داخله.

قاعدة ذهبية للمناقشة

لذلك فصلنا خدمة مطابقة أسماء الأدوية؛ النتيجة بحث أدق؛ لكن القرار النهائي للمخزون يبقى عند الباك وقاعدة البيانات. ابدأ دائما بالمشكلة، ثم القرار الهندسي، ثم الفائدة، ثم القيد. مثال: احتجنا بحثا يتحمل أخطاء الكتابة؛

قراءة ثانية — 45 دقيقة

على البحث، سلسلة التوريد، التبرعات، وخدمات الذكاء الثلاث. ركز

قراءة أولى — 35 دقيقة

الأدوار، المعمارية، والسيناريوهات الرئيسية لتكوين الصورة الكبيرة. اقرأ الملخص،

قبل العرض — 10 دقائق

راجع نص العرض المختصر، القيود، وخطة العرض التجريبي.

تدريب — 30 دقيقة

عال عن أسئلة اللجنة في آخر الملف دون النظر إلى الإجابات. جاوب بصوت

خريطة الدليل

القسم	ما الذي ستفهمه؟
1-4	الفكرة العامة، المستخدمون، المعمارية، والتقنيات
5-12	البحث، الوصفات، التبرعات، التوريد، والخصوصية
13-18	خدمات الذكاء، Hybrid RAG، البدائل، والتنبؤ
19-23	الأمن، الاعتمادية، الاختبار، والقيود
24-27	عرض المناقشة، الأسئلة المتوقعة، والمصطلحات

1. المشروع في دقيقة

جوابك المختصر عندما تسألك اللجنة: ما هو مشروعكم؟

الجواب الجاهز

ومساعد صيدلاني يعتمد على مصادر مسترجعة من قاعدة معرفة. الذكاء هنا داعم للقرار وليس بديلا عن الطبيب أو الصيدلي. عبر سلسلة توريد قابلة للتتبع. كما تضيف ثلاث خدمات ذكاء مساعدة: التحقق الأولي من التراخيص، مطابقة أسماء الأدوية، المستخدم على البحث عن الدواء وأقرب مكان متاح، وتدير الوصفات والطلبات والتبرعات، وتربط الصيدليات بالمستودعات دوائي منصة صحية رقمية تربط المستخدم والصيدلية والمستودع والمندوب والمنظمة والإدارة ضمن نظام واحد. تساعد

المشكلة التي يحلها المشروع

- المستخدم قد يعرف الاسم التجاري بشكل خاطئ أو لا يعرف أين يتوفر الدواء.
- الصيدلية تحتاج إدارة مخزون ووصفات وطلبات وتوريد ضمن مكان واحد.
- المستودع يحتاج دفعات وصلاحيات وشحنات وفواتير ومرتجات قابلة للتدقيق.
- التبرعات الدوائية تحتاج رقابة صيدلانية ومسارا واضحا بين المتبرع والمنظمة والصيدلية.
- الإدارة تحتاج إشرافا وحوكمة من دون كشف تفاصيل مالية أو تشغيلية لا تحتاجها.

القيمة الأساسية

للصيدلية

إدارة المخزون والطلبات والوصفات والتبرعات وساعات العمل.

للمستخدم

للمعلومة والدواء، مع بحث متسامح مع الأخطاء وموقع جغرافي. وصول أسرع

لِلإدارة

اعتمادات، سجلات تدقيق، مؤشرات مجمعة، وضبط صلاحيات.

للمستودع

توريد منظم بالدفعات، تتبع شحنات، فواتير، مرتجات واستدعاءات.

2. أصحاب المصلحة والأدوار

المشروع Role-Based؛ كل دور يرى ما يحتاجه فقط.

الدور	أهم المسؤوليات
المستخدم	بحث، وصفة ذكية، طلبات، تبرعات، ملف طبي، طوارئ
الصيدلية	مخزون، طلبات أدوية، وصفات محجوزة، تحقق تبرعات، توريد
المنظمة	حملات، عروض تبرع، طلبات مساعدة، تحقق وتنظيم التوزيع
المستودع	دفعات، طلبات، شحنات، مناديب، فواتير، مرتجعات، استدعاء
المندوب	استلام المهام، خط السير، إثبات التسليم، سجل العمليات
المدير	اعتمادات، حسابات، سجل تدقيق، دليل أدوية، مؤشرات إشرافية

لماذا فصل الأدوار مهم؟

لذلك المدير يراقب سلسلة التوريد بمؤشرات مجمعة، بينما تفاصيل الفاتورة والمرجع المالي تبقى لدى الأطراف التشغيلية. المالية ليست متساوية الحساسية. مبدأ أقل صلاحية يعني أن كل حساب يحصل فقط على الأفعال والبيانات اللازمة لعمله. لأن بيانات الوصفة والحالة الصحية والدفعات

مثال صلاحيات

- المستخدم ينشئ طلب دواء ويرى طلباته فقط.
- الصيدليات القريبة ترى الطلب ضمن نطاقها ويمكنها الرد حسب التوفر.
- المستخدم يختار الرد المناسب، ولا يحصل طرف آخر على ملفه الطبي الكامل.
- المدير يرى عدد الطلبات وحالتها العامة لأغراض الحوكمة، لا محادثة المستخدم الخاصة.

كيف تتعاون واجهات الويب والموبايل والباك وخدمات الذكاء؟

1 واجهة React أو تطبيق Flutter يرسل طلب HTTPS إلى ASP.NET Core API.

2 ال API يتحقق من هوية المستخدم والدور وصحة المدخلات.

3 طبقة التطبيق تنفذ منطق العمل وتتعامل مع قاعدة البيانات عبر EF Core.

4 عند الحاجة فقط، يستدعي الباك خدمة ذكاء داخلية بمفتاح خدمة ووقت انتظار محدد.

5 النتيجة تراجع وتربط ببيانات النظام ثم تعاد ك JSON منظم للواجهة.

API / Application

مصادقة، صلاحيات، تحقق، وتنفيذ السيناريوهات ASP.NET Core. هو بوابة الثقة.

Presentation

استخدام ولا يحتويان أسرار الخدمات Flutter للويب و React للموبايل؛ عرض وتجربة.

AI Microservices

يمكن تطويرها ونشرها ومراقبتها منفصلة Python/FastAPI. لخدمات متخصصة.

Domain / Data

البيانات؛ هي مصدر الحقيقة للمخزون والطلبات والحسابات. كيانات وقواعد العمل وقاعدة

نقطة مهمة

يبقى في الخادم. كذلك خدمة الذكاء لا تقرر أن الدواء متوفر؛ هي تفهم أو تطابق، ثم الباك يسأل قاعدة البيانات الفعلية. الواجهة لا تتصل بـ Gemini أو بخدمات الذكاء مباشرة. المفتاح

4. التقنيات ولماذا اخترناها

التقنية	الدور والسبب
React + Vite	واجهة مكونات سريعة التطوير مع بناء محلي سريع
TanStack Query	جلب وتخزين مؤقت وإعادة تحديث البيانات من الخادم
Tailwind CSS	تنسيق متجاوب وموحد عبر الشاشات
React Router	مسارات وحماية صفحات حسب الدور
i18next	العربية والإنجليزية والتركية مع RTL/LTR
ASP.NET Core	أمان وأداء، Middleware، حقن اعتماديات API
EF Core	ربط الكيانات بقاعدة البيانات وهجرات وعلاقات
FastAPI	واجهات Python سريعة ومناسبة لنماذج الذكاء
Redis	ذاكرة سريعة للكاش والجلسات وحدود المعدل
Docker Compose	تشغيل خدمات الذكاء واعتمادياتها بشكل متكرر
Flutter	تطبيق Android/iOS من قاعدة كود واحدة

لماذا Microservices للذكاء؟

الأعمال. فصلها يمنع ربط الباك بتقنية واحدة، ويسمح بتوسيع خدمة معينة أو استبدال نموذجها دون إعادة كتابة المنصة. لأن مكتبات OCR و NLP والنماذج أثقل وأسرع تغييرا من منطق

فكر بالمشروع كمجموعات مترابطة، لا كجدول واحد ضخم.

الدواء والمخزون Medicines, Aliases, InventoryItems, MedicineBatches, الصور والترجمات.	الهوية والحسابات Users, Roles, والمندوب والمستودع والمنظمة والصيدلية وملفات المستخدم.
التبرعات Campaigns, DonationOffers, OfferImages, AssistanceRequests.	الطلبات والوصفات MedicineRequests, PrescriptionOrders, PrescriptionItems, التذكيرات.
المال والمرتجعات Invoices, Payments, Returns, ReturnBatchAllocations, Recalls.	التوريد SupplyOrders, Items, BatchAllocations, Shipments, TrackingEvents.
الحوكمة Notifications, AuditLogs, Verifications, SOS, SearchHistory.	المحادثة مع فصل جلسات المستخدمين ChatSessions, ChatMessages.

قرار تصميم مهم

أن يملك عدة تخصيصات، وكل تخصيص يحمل كمية محجوزة ومسلمة من دفعة محددة. وينعكس ذلك أيضا على المرتجعات. الكمية موزعة على أكثر من Batch. لذلك يوجد جدول SupplyOrderItemBatchAllocations: السطر الواحد في الطلب يمكن ربط صنف الطلب بدفعة واحدة لا يكفي عندما تكون

6. تسجيل الدخول والأمن والخصوصية

- المصادقة تحدد من المستخدم؛ التفويض يحدد ماذا يستطيع أن يفعل.
- API تحسن تجربة الواجهة، لكن الحماية الحقيقية دائماً داخل Route Guards
- التحقق من المدخلات يتم بقواعد واضحة قبل تنفيذ منطق العمل.
- المفاتيح مثل Gemini و Google Places تبقى في الإعدادات السرية/البيئة ولا ترسل للمتصفح.
- تسجيل العمليات الحساسة لنعرف من فعل ماذا ومتى وهل نجحت العملية Audit Logs
- صور التبرعات والوثائق الحساسة تعرض عبر endpoints محمية، لا كروابط عامة بلا صلاحيات.
- الاستجابات لا تعيد حقولاً أكثر من حاجة الشاشة؛ وهذا يقلل التسريب العرضي.

الخصوصية حسب الدور

إجماليات ومعدلات، بينما أسماء الأطراف والمبالغ والمراجع وأسباب المرتجعات تبقى في نطاق المستودع والصيدلية المخولين. المشرف يحتاج مراقبة الصحة العامة للنظام، لا التجسس على تفاصيل الأطراف. لذلك تعرض لوحة مراقبة التوريد

دفاع متعدد الطبقات

- 1 صالح HTTPS + Token
- 2 سياسة Role/Policy على endpoint
- 3 التحقق من ملكية السجل داخل الخدمة
- 4 للمدخلات والهجوم Validation
- 5 سجل تدقيق واستجابة خطأ لا تكشف أسراراً

7. كيف تتم عملية البحث عن الدواء؟

7 البحث عن الدواء

1 المستخدم يكتب الاسم بالعربية أو الإنجليزية، وقد يحتوي خطأ إملائيًا أو تبديل لوحة مفاتيح.

2 خدمة Drug Search تنظف النص وتبحث بالاسم والمرادفات والتشابه واللفظ والتصحيح المحتمل.

3 تعيد Candidates مرتبة مع درجات ثقة؛ لا تعلن وجود مخزون من تلقاء نفسها.

4. في قاعدة البيانات Medicine/Alias يطابق المرشح مع ASP.NET.

5 ثم يستعلم عن InventoryItems والصيدليات المسموح عرضها وموقع المستخدم.

6 تعود النتائج الفعلية مع الاسم والكمية/التوفر والمسافة والتفاصيل المناسبة.

Fuzzy Match

يتحمل اختلاف حرف أو ترتيبا بسيطا.

Exact Match

الاسم مطابق بعد التطبيع؛ أعلى ثقة وأقل مخاطرة.

Phonetic/Alias

يستفيد من اللفظ والمرادفات والأسماء التجارية/العلمية.

Keyboard Fix

يحاول اكتشاف كتابة كلمة عربية بلوحة إنجليزية أو العكس.

أهم هنا Precision

فاتتنا بعض النتائج. لذلك تكون العتبات محافظة، والمرشح غير الواثق يعرض كاقترح أو يطلب توضيحا بدلا من اعتباره دواء مؤكدا. في الدواء نفضل ألا نعرض نتيجة خاطئة بثقة عالية، حتى لو

8. GPS وHaversine وGoogle Places وOSRM

1. يعطي إحداثيات المستخدم بعد منحه الإذن GPS.
2. قاعدة البيانات تعيد الصيديات المسجلة وإحداثياتها.
3. معادلة Haversine تحسب المسافة الهوائية التقريبية بسرعة لترتيب/تصفية المرشحين.
4. يضيف صيديات خارجية قريبة غير مسجلة في المنصة عند تفعيل المفتاح Google Places.
5. يحسب طريق القيادة الفعلي والمسافة والزمن المتوقع للمرشحين الأهم OSRM.
6. إذا فشل مزود خارجي، تبقى نتائج المنصة الداخلية متاحة مع fallback واضح.

الفرق الذي يجب قوله

عند عرض الطريق Route يعرف شبكة الطرق ويعطي OSRM. لا يعرف الشوارع؛ هو خط تقريبي على سطح الأرض Haversine واقعية. نستخدم الأول للسرعة في الفرز، والثاني للدقة.

سبب خطأ REQUEST_DENIED في Google Places

- المفتاح غير صحيح أو Places API غير مفعلة على المشروع.
- غير مفعّل أو قيود المفتاح لا تسمح بالخادم/العنوان الحالي Billing.
- الحل ليس كشف المفتاح في الفرونت؛ بل تصحيح إعدادات الخادم والاحتفاظ بال fallback.

9. رحلة المستخدم والمستخدم الذكية

سيناريو طلب دواء

- 1 يبحث المستخدم؛ إن لم يجد توفرا مناسباً ينشئ طلب دواء.
- 2 يحفظ الباك الطلب ويربطه بالمستخدم والموقع والدواء المطلوب.
- 3 تظهر المهمة للصيديات المناسبة ويمكنها الرد حسب المخزون.
- 4 يختار المستخدم العرض، وتحدث الحالة وتصل إشعارات للأطراف.

سيناريو الوصفة

- 1 يرفع المستخدم صورة الوصفة.
- 2 خدمة الاستخراج تحاول قراءة البوند Structured Output؛ OCR محلي fallback عند الحاجة.
- 3 تعرض البوند للمستخدم للمراجعة والتصحيح قبل الاعتماد.
- 4 ينشأ PrescriptionOrder وتتعامل الصيدلية مع الحجز والتجهيز.

حدود طبية

تشخيصا. النص المستخرج يحتاج مراجعة بشرية، والصيدلي مسؤول عن مطابقة الوصفة النظامية والدواء والجرعة قبل الصرف.
قراءة الوصفة آليا ليست وصفا علاجيا ولا

10. سيناريو التبرع والمساعدة

- 1 المنظمة الموثقة تنشئ حملة وتحدد المنطقة والمدة والأدوية المطلوبة.
- 2 المستخدم يختار التبرع للحملة ويدخل الدواء والكمية والصلاحيات ويرفع صورة واضحة.
- 3 الصورة تمنح الصيدلية فحصاً أولياً للعبوة والاسم والصلاحيات والحالة الظاهرة.
- 4 تراجع الصيدلية العرض مبدئياً؛ ثم يحضر المتبرع الدواء للفحص الفيزيائي.
- 5 بعد التحقق النهائي تحدث الحالة ويسجل الاستلام وتتابع المنظمة التوزيع.

الصورة ليست موافقة نهائية

للفحص فقط. القبول النهائي يتم لدى صيدلية وبوجود مختص، مع رفض الأدوية غير الصالحة أو المفتوحة أو مجهولة التخزين. لا يمكن للصورة تأكيد ظروف التخزين أو سلامة العبوة الداخلية. لذلك هي مرشح

طلب المساعدة

حاجة دوائية من مستخدم/جهة مع منطقة وحالة متابعة.

عرض التبرع

من قدم، ما الدواء، الكمية، الصلاحيات، الصور، الجهة، والحالة.

التفاصيل

تفتح بسجل محدد وصلاحيات صحيحة، وليست مجرد بطاقة عامة.

الحملة

هدف معلن ومدة وأدوية مطلوبة وتنظيم لاستقبال العروض.

11. سلسلة التوريد من الطلب إلى التسليم

11 سلسلة التوريد

- 1 المصدلية تنشئ Supply Order بأصناف وكميات.
- 2 المستودع يراجع الطلب ويقبل أو يرفض مع سبب.
- 3 أثناء التجهيز تحجز الكميات من الدفعات المناسبة وتسجل Batch Allocations.
- 4 ينشئ المستودع Shipment ويسننها إلى مندوب متاح.
- 5 المندوب يحدث حالات التحميل والخروج والوصول، ويمكن تسجيل Tracking Events.
- 6 عند التسليم تؤكد الكميات الفعلية وتغلق/تجزأ الحالة حسب النقص.
- 7 تدار الفاتورة والمدفوعات بين الجهتين وتبقى قابلة للتدقيق.

المجال	أمثلة حالات
الطلب	Submitted → Accepted → Preparing → Ready → Delivered
الشحنة	Created → Assigned → Loading → OutForDelivery → Delivered
الفاتورة	Draft → Issued → PartiallyPaid → Paid / Overdue
المرتجع	Requested → Approved → Collected → Completed
الاستدعاء	Active → Completed / Cancelled

12. الدفعات المتعددة و FEFO والمرتجعات

مثال دفعتين

واحدة، ننشئ تخصيصين: 20 من B100 و 10 من B200. مجموع التخصيصات يجب أن يساوي الكمية المحجوزة/المسلمة. 30 علب. الدفعة B100 فيها 20 علبه وصلاحيتهما أقرب، والدفعة B200 فيها 10 علب. بدلا من إجبار OrderItem على Batch طلبت الصيدلية

لماذا FEFO؟

صالحة وغير محجوزة بالكامل. الهدف تقليل الهدر مع الحفاظ على شروط السلامة: First Expired, First Out: FEFO. نفضل الدفعة الأقرب انتهاء بشرط أن تكون

- مكان التخزين داخل المستودع يجب أن يكون معروفا للوصول للدفعة فعليا.
- الحجز يمنع بيع الكمية نفسها لطلبين متزامنين.
- التسليم الفعلي قد يختلف عن المطلوب؛ لذلك نخزن Delivered Quantity.
- المرتجع يربط بتخصيص الدفعة الأصلي حتى نعرف من أي Batch خرجت العبوات.
- إعادة الكمية للمخزون لا تتم عشوائيا؛ تحتاج اكتمال المراجعة وحالة مناسبة.

التزامن

القديمة لحماية المخزون، ثم يطلب تحديث القائمة وإعادة المحاولة. هذا Optimistic Concurrency وليس عطل واجهة. عند تغير البيانات بين فتح النافذة وتأكيده العملية، يرفض الخادم النسخة

13. الفواتير، المرتجعات، الاستدعاءات والتقارير

الفاتورة

والمبالغ وحالة الدفع؛ الوصول التفصيلي للأطراف المخولة.
تعكس الطلب والشحنة

الدفعة المالية

تسجل المبلغ والمرجع والحالة؛ لا تعرض كمعلومة عامة للمدير.

المرتجع

يربط السبب والكمية والدفعة الأصلية ومسار الاستلام والمعالجة.

الاستدعاء

يحدد دفعات أو دواء متأثر وبنية الجهات التي استلمته لتتخذ إجراء.

ماذا يرى المدير؟

- أعداد الطلبات والشحنات حسب الحالة والفترة.
- نسب التسليم والطلبات المتأخرة ومعدل الإرجاع.
- اتجاهات تشغيلية ورسوم بيانية مجمعة تكشف الاختناق.
- لا يرى تلقائياً أسماء الأطراف أو المبالغ أو مرجع الدفع أو الملاحظات الخاصة.

لماذا التقارير؟

فلا يساعد المدير، لذلك تستخدم الرسوم للاتجاهات، والجداول للتفاصيل التشغيلية، و CSV/PDF للمشاركة والأرشفة.
يحول البيانات إلى قرار: هل ارتفع التأخير؟ أين تزيد المرتجعات؟ ما الصنف الذي يقترب من النفاد؟ أما جدول طويل بلا مؤشرات التقرير الجيد

14. خدمات الذكاء الاصطناعي الثلاث

الخدمة	مهمتها	ما لا تفعله
License AI	ومقارنة اسم الرخصة OCR	لا تعتمد الصيدلية نهائيا
Drug Search	تطبيع ومطابقة أسماء الأدوية	لا تقرر التوفر أو الصرف
Smart Bot	وصفة، بدائل، نفاذ، RAG	لا يشخص ولا يستبدل المختص

لماذا ثلاث خدمات؟

أثقل، الفصل يجعل الاختبار والتوسع والتعطل مستقلا. إذا تعطلت خدمة الذكاء لا يجب أن تتوقف العمليات الأساسية للمنصة. كل مشكلة مختلفة: الرخصة صورة ووثيقة؛ البحث يحتاج مطابقة لغوية سريعة؛ المساعد يحتاج استرجاعا وتوليدا ونماذج

التشغيل المحلي

الترخيص 8001، البحث 8002، Redis داخلي. تستخدم health endpoints لمعرفة هل الخدمة Started فقط أم Ready فعلا. من مجلد `services_AI: docker compose up --build -d` ثم `docker compose ps`. المنافذ الافتراضية: المساعد 8000،

مفتاح الخدمة

X-API-Key للخدمة الداخلية. Gemini key يبقى داخل بيئة خدمة المساعد. لا يوضع أي مفتاح في React أو Flutter أو Git. يرسل ASP.NET ترويسة

15. ما هو Hybrid RAG وكيف يعمل الشات بوت؟

مرتبطة بالسؤال من قاعدة معرفة ثم نطلب منه بناء جواب مقيد بها مع مصادر ، RAG = Retrieval-Augmented Generation ، أي أن النموذج لا يجيب من ذاكرته العامة فقط؛ بل نسترجع مقاطع

- 1 تنظيف سؤال المستخدم وتحديد اللغة والسياق والسلامة.
- 2 والبحث عن تقارب المعنى Embeddings تحويل السؤال والمقاطع إلى Dense Retrieval:
3. بحث كلمات كلاسيكي ممتاز للأسماء والعبارات الدقيقة: BM25
4. من دون افتراض أن درجاتهما على المقياس نفسه BM25 و Dense يدمج ترتيب RRF
5. ترفع المطابقة المباشرة للاسم أو الحقل المهم Exact-field boosts
6. يقرأ السؤال مع كل مقطع مرشح ويعيد ترتيب النتائج بدقة أعلى CrossEncoder
- 7 إذا تجاوزت الثقة الحد، ترسل أفضل المقاطع إلى Gemini مع مخطط Structured Output.
- 8 يعيد النظام جواباً منظماً ومراجعاً مرقمة؛ وإن كانت الثقة ضعيفة يطلب توضيحاً أو يمتنع.

لماذا Hybrid؟

عندما يكتب المستخدم اسماً دقيقاً مثل Acarbose. الجمع يعطي تغطية أفضل، وإعادة الترتيب تقلل النتائج غير الملائمة. البحث الدلالي يفهم أن «نفسى مقطوع» قريب من «ضيق تنفس»، لكن BM25 أقوى

16. المصادر و CrossEncoder و RRF و Structured Output

Embedding

يمثل معنى النص؛ النصوص المتقاربة معنى تكون متقاربة رياضياً.
متجه أرقام

BM25

ترتيب كلمات يأخذ تكرار الكلمة وندرتها وطول الوثيقة بالحسابات.
محرك

RRF

عدة محركات: النتيجة التي تظهر مبكراً في أكثر من قائمة ترتفع.
يجمع مراتب

CrossEncoder

وأبطأ يقرأ السؤال والمقطع معاً؛ يستخدم فقط على قائمة قصيرة.
نموذج أدق

Structured Output

على JSON مطابق لمخطط Pydantic بدلاً من نص عشوائي.
إجبار النموذج

أرقام المصادر

[1] يربط الادعاء بالمقطع الأول المسترجع كي يمكن التحقق منه.
الرمز

معادلة RRF المبسطة

الوثيقة بالمرتبة الأولى تحصل على مساهمة أكبر من المرتبة العاشرة. $k=60$ يخفف الفروق الحادة ويجعل الدمج مستقرًا.
لكل وثيقة نجمع $1 \div (k + \text{rank})$ من كل قائمة.

Recall مقابل Precision

الموجودة، كم واحدة استرجعنا؟ في سياق طبي نميل إلى Recall: من النتائج التي عرضناها، كم واحدة صحيحة؟ Precision: معقول عبر البحث الهجين Recall أعلى كي لا نقدم معلومة خاطئة بثقة؛ مع الحفاظ على Precision من كل النتائج الصحيحة.

لماذا هو موجود؟ Redis: 17.

- سريع في الذاكرة، وليس بديلا عن قاعدة البيانات الأساسية Key-Value مخزن Redis.
- يخزن مؤقتا نتائج متكررة أو بيانات جلسة قصيرة لتقليل الزمن والحمل.
- يساعد على فصل ذاكرة كل جلسة Chat ومنع اختلاط المستخدمين.
- يمكن استخدامه لحدود المعدل Rate Limiting حتى لا يسيء عميل واحد استخدام الخدمة.
- البيانات الدائمة مثل الطلبات والفواتير والمخزون تبقى في قاعدة النظام، لا في Redis وحده.

الكاش الصحيح

عند غياب Redis يمكن للخدمة العمل بقدرات أقل أو تخزين محلي حسب الإعداد، لكن الإنتاج يستفيد من Redis المشترك. يجب أن يكون له مدة صلاحية ومفتاح واضح وآلية إبطال عند تغير الأصل. الكاش قديم ليس مصدرا للحقيقة.

كيف نحسن زمن الشات؟

- نحسب Embeddings للقاعدة مرة ونخزنها، ولا نعيدها لكل سؤال.
- نستخدم Dense وBM25 لاسترجاع قائمة صغيرة ثم CrossEncoder عليها فقط.
- نعمل streaming ليرى المستخدم بداية الجواب قبل اكتماله.
- نضبط timeouts وhealth checks ونعطي fallback واضحا عند تعطل Gemini.

18. بدائل الأدوية وتوقع النفاذ

منطق البدائل

- المطابقة المحافظة تتطلب نفس المادة الفعالة والتركيز والشكل الصيدلاني.
- النتيجة Candidate بديل دوائي، وليست إذنا تلقائيا بالاستبدال.
- الحالات الحساسة تحتاج مراجعة صيدلي/طبيب وقواعد إضافية قبل الصرف.

معادلة الطلب اليومي

المعادلة

آخر 7 أيام ÷ 7 + 20% × (مبيعات آخر 30 يوما ÷ 30). إذا غاب أحد المدخلات تعاد معايرة الأوزان المتبقية لتجمع 100%.
الطلب اليومي = 45% × المتوسط اليومي المرسل + 35% × (مبيعات

حتى النفاذ ≈ 10 أيام. والكمية المستهدفة في التنفيذ الحالي تغطي 37 يوما تقريبا، أي 30 يوما + مخزون أمان 7 أيام.
= 330 أي 11/يوم. الطلب الموزون = $0.20 \times 11 + 0.35 \times 14 + 0.45 \times 10 = 11.6$ علبة/يوم. إذا المخزون 116 علبة فالأيام
مثال: متوسط عام 10 علب/يوم، آخر 7 أيام = 98 علبة أي 14/يوم، وآخر 30 يوما

مهم جدا للمناقشة

خيارات مستقبلية عندما تتوفر بيانات تاريخية كافية وتقييم زمني صحيح، وليست أسماء نذكرها على أنها تعمل الآن.
Decision شفاف وليس نموذج Machine Learning مدربا. Prophet وXGBoost وLSTM وTemporal Fusion Transformer هذا Support Algorithm

19. متى نستخدم Prophet أو XGBoost أو LSTM أو TFT؟

المتى يناسب؟	الفكرة العامة	الخيار
سلسلة واضحة وبيانات متوسطة	يرسم اتجاهها ومواسم وعطلا	Prophet
عروض، أسعار، أيام، مخزون، فروع	يتعلم من خصائص مصممة يدويا	XGBoost
تاريخ طويل ونظيف لكل صنف	شبكة تتعلم التسلسل عبر الزمن	LSTM
نظام كبير وتفسير أهمية المتغيرات	يتعامل مع عدة سلاسل وخصائص	TFT
بيانات قليلة وبداية آمنة	معادلة واضحة قابلة للشرح	Decision Support

لماذا بدأنا بالمعادلة؟

- لا تحتاج آلاف السجلات ولا GPU.
- يمكن تفسير كل نتيجة للصيدلي والمناقش.
- سهولة الاختبار ووضع حدود أمان حولها.
- تبني baseline نقارن به أي نموذج مستقبلي؛ إذا لم يتفوق النموذج عليه فلا قيمة لتعقيده.

التقييم الصحيح

على فترة لاحقة، لا نخلط الأيام عشوائيا. نقيس MAE/WAPE، ونقيس أيضا أثرا عمليا: كم حالة نفاد أو تخزين زائد تم تجنبها؟
نقسم البيانات زمنيا: ندرب على الماضي ونختبر

20. خدمة التحقق الأولي من الترخيص

20 التحقق من الترخيص

1 ترفع الصيدلية صورة وثيقة الترخيص وبيانات التسجيل.

2 الخدمة تعالج الصورة وتطبق OCR لاستخراج النص.

3 تبحث عن اسم حامل الترخيص/البيانات المهمة وتقارنها بالاسم المسجل.

4 تعيد نتيجة مساعدة مع درجة/حالة وملاحظات فنية.

5 المدير يراجع الوثيقة والنتيجة ويقرر اعتمادا أو رفضا أو مراجعة يدوية.

لماذا لا نستخدم آليا؟

والأختام واحتمال التلاعب تجعل OCR أداة فرز لا سلطة قانونية. القرار النهائي بشري ويحفظ سبب القرار في سجل التدقيق. جودة الصورة والخط

- توجد إعادة محاولة لأن OCR قد يكون بطيئا أو الخدمة في طور التحميل.
- يجب تحديد حد حجم الملف وأنواع الصور والتحقق منها.
- لا نرسل الوثيقة إلى واجهة غير مخولة ولا نجعل الرابط عاما.

21. الويب والموبايل: نفس المنطق، تجربتان

الموبايل Flutter

أثناء الحركة: موقع، إشعارات، مسح، ملفات، وعمليات يومية. تجربة

الويب React

لوحات كاملة للأدوار، جداول وتقارير وإدارة عمليات واسعة.

استجابة وتوطين

متجاوبة، العربية RTL واللغات الأخرى LTR، وأرقام قابلة للقراءة. واجهات

موحد API

تتصلان بنفس API ASP.NET؛ فلا نكرر منطق العمل الحساس. المنصتان

الربط مع الباك

الحاسوب؛ يجب استخدام عنوان الخادم المتاح من الهاتف أو نطاق HTTPS مثل إعداد النشر، مع CORS وشهادة صحيحة. عنوان الـ API يحدد من إعداد بيئي وليس نصا موزعا في الصفحات. على جهاز حقيقي لا يعني localhost

إدارة الحالة والبيانات

- ضمن مدة محددة cache في الويب يمنع الطلبات المكررة ويعيد استخدام TanStack Query
- Riverpod/GoRouter/Dio مع data/repository/presentation يستخدم طبقات Flutter
- بعد mutation ناجحة نحدث أو نبطل query المعنية فقط، لا نعيد تحميل التطبيق كله.

22. كيف نعالج البطء والأخطاء؟

المشكلة	المعالجة
صفحة تنتظر عدة طلبات	تشغيل مستقل/متواز، تجميع endpoint، skeletons
طلبات تتكرر عند التنقل	Cache و deduplication و staleTime
مزود خارجي يتعطل	رسالة قابلة للفهم + fallback + Timeout
خدمة AI تبدأ ببطء	قبل توجيه الطلبات health readiness
تضارب مخزون	transaction + concurrency check + refresh
قائمة كبيرة	على الخادم pagination/filtering
صورة ضخمة	حد حجم، ضغط، تحقق نوع ورفع منظم

قاعدة

بيانات أكثر من الحاجة، retry غير محدود، أو query تعاد بسبب مفتاح غير ثابت. القياس من Network و logs يسبق التخمين. شاشة Loading طويلة لا تعني دائماً شبكة بطيئة؛ قد يكون هناك طلب متسلسل، endpoint يجلب

جيد Fallback

وعند فشل Gemini لا نخلق جواباً؛ نخبر المستخدم أن الخدمة غير متاحة ونبقى البحث والعمليات الأساسية شغالة. عند فشل Google Places مثلاً لا نمسح نتائج الصيدليات الداخلية.

23. خطة اختبار وعرض أمام اللجنة

23 الاختبارات والعرض التجريبي

اختبارات أساسية

- للمعادلات والحالات والتحقق من المدخلات Unit Tests
- وقاعدة البيانات والصلاحيات API لل Integration Tests
- بصمت JSON كي لا يتغير Python وخدمات ASP.NET بين Contract Tests
- للمسارات المهمة: تسجيل، بحث، طلب، توريد، تبرع UI Tests
- منتهي token، دور خاطئ، سجل ليس ملك المستخدم، ملف ضخم: Security Tests
- Performance Tests مع cold start وwarm cache لل AI للقوائم والبحث وال

سيناريو Demo آمن

- 1 سجل الدخول كمستخدم وابحث باسم دواء مكتوب بخطأ بسيط.
- 2 اعرض الصيدليات الداخلية والمسافة، ثم صفحة التفاصيل.
- 3 انتقل لصيدلية وأنشئ طلب توريد صغيرا.
- 4 انتقل للمستودع واقبل الطلب وخصمه من دفعتين.
- 5 أسند شحنة لمندوب وأظهر تغير الحالة.
- 6 اختتم بسؤال واضح للشات مع ظهور المصادر، ثم اشرح حدود السلامة.

خطة بديلة

للخطوات، وشغل health checks قبل العرض. لا تجعل نجاح المناقشة معتمدا بالكامل على Google/Gemini أو شبكة القاعة. حضر بيانات seeded وصورا محلية ولقطات

24. نقاط القوة والحدود بصراحة

24. نقاط القوة والحدود

نقاط القوة

- منصة متعددة الأدوار تغطي رحلة صحية وتشغيلية مترابطة.
- مصدر حقيقة مركزي وصلاحيات على مستوى الخادم.
- بحث لغوي متسامح مع الأخطاء وموقع جغرافي بمستويات دقة.
- سلسلة توريد تدعم عدة دفعات وتتبعها ومرتجات واستدعاءات.
- ذكاء مفصول وقابل للاستبدال مع مراجع وبنية Structured Output.
- تصميم يحافظ على دور المختص البشري في القرارات الطبية والقانونية.

حدود واقعية

- دقة المعرفة الطبية مرتبطة بجودة وحدثا وترخيص المصادر المتاحة.
- الأسماء التجارية السورية تحتاج بيانات محلية ونشرات رسمية ومراجعة صيدلي.
- خدمات خارجية قد ترفض أو تتعطل أو تتغير تكلفتها Gemini و Google Places.
- توقع النفاذ الحالي خوارزمية شفافة لا نموذجا متعلما موسميا.
- والصور لا يثبتان وحدهما صحة الوثيقة أو سلامة الدواء OCR.
- النشر الحقيقي يحتاج مراقبة ونسخا احتياطية وإدارة أسرار وسياسة احتفاظ بالبيانات.

الإجابة الناضجة

ذكر القيود لا يضعف المشروع؛ يثبت أنك تفهم حدود النظام وتعرف ما يلزم لنقله من مشروع تخرج إلى منتج صحي مسؤول.

25. أسئلة متوقعة من اللجنة

س: لماذا لم تضعوا كل شيء داخل ASP.NET؟

ج: الذكاء يعتمد منظومة Python ونماذج مختلفة؛ فصلها يقلل الترابط ويسمح بالتوسع والاستبدال، مع بقاء الباك بوابة الثقة. ج: لأن خدمات

س: هل الشات يشخص المرض؟

ج: لا. يقدم معلومات صحية عامة مرتبطة بمصادر ويطبق تنبيهات سلامة، ويوجه للطوارئ/المختص عند الحاجة.

س: لماذا RAG بدل Gemini وحده؟

ج: لتقييد الجواب بمعرفة مسترجعة، إظهار المصادر، وتحديث المحتوى دون إعادة تدريب النموذج.

س: لماذا بحثان Dense وBM25؟

ج: Dense يفهم المعنى، وBM25 يلتقط الأسماء والكلمات الدقيقة؛ RRF يجمعهما وCrossEncoder يحسن الترتيب.

س: كيف تمنعون بيع نفس المخزون مرتين؟

ج: بمعاملات قاعدة بيانات، حجز كمية على الدفعة، والتحقق من التزامن قبل التأكيد.

س: لماذا جدول تخصيص دفعات؟

ج: لأن صنف طلب واحد قد يلبي من عدة دفعات؛ العلاقة ليست واحدا لواحد.

س: هل البديل الدوائي آمن تلقائيا؟

ج: هو مرشح يحافظ بنفس المادة والتركيز والشكل، لكن الصرف النهائي بقرار مختص وقواعد طبية.

س: كيف تحمون مفاتيح API؟

ج: توجد في الخادم/بيئة النشر أو secrets، ولا تصل إلى React أو Flutter ولا ترفع إلى Git.

س: ماذا يحدث إذا تعطلت خدمة خارجية؟

ج: Timeout ورسالة واضحة وfallback؛ لا تتوقف وظائف المنصة الأساسية ولا نخترع نتائج.

س: كيف تقيسون النجاح؟

ج: تقنيا بزمان الاستجابة والأخطاء ودقة البحث، وعمليا بزمان العثور على الدواء، نجاح الطلبات، تقليل النفاد والتأخير.

26. نص جاهز لعرض 5-7 دقائق

0:45-0:00 — المشكلة

توفر الدواء، وتعمل الصيدليات والمستودعات والمنظمات غالباً بأنظمة منفصلة. هدفنا ربط الرحلة كاملة ضمن منصة واحدة. يعاني المستخدم من صعوبة معرفة

1:30-0:45 — الحل

دوائي تخدم ستة أدوار: مستخدم، صيدلية، منظمة، مستودع، مندوب ومدير. لكل دور لوحة وصلاحيات وسيناريوهات واضحة.

2:30-1:30 — المعمارية

الباك يدير الهوية ومنطق الأعمال وقاعدة البيانات، ويستدعي ثلاث خدمات ASP.NET Core API يتصلان بـ React وFlutter. متخصصة عند الحاجة Python.

3:30-2:30 — سيناريو عملي

أعرض البحث باسم خاطئ، مطابقة الدواء، الصيدليات القريبة، ثم طلباً يتدرج عبر الصيدلية أو سلسلة التوريد.

4:45-3:30 — الذكاء

أشرح Hybrid RAG: Dense وBM25، دمج RRF، إعادة ترتيب CrossEncoder، ثم Gemini مع Structured Output ومصادر.

5:45-4:45 — الأمان

أكد أن الذكاء داعم لا مقرر، وأن المفاتيح بال خادم والصلاحيات في API، وأن المدير يرى مؤشرات مجمعة.

6:30-5:45 — القوة والحدود

أذكر الدفعات المتعددة والتبرع بصور وفحص صيدلي، ثم أذكر اعتماد بعض الخدمات على مزود خارجي وخطة fallback.

7:00-6:30 — الخلاصة

شاشة بحث فقط؛ هو شبكة صحية وتشغيلية تحافظ على مصدر حقيقة واحد وتضيف الذكاء حيث يعطي قيمة قابلة للضبط. المشروع ليس

27. قاموس سريع بالمصطلحات

المصطلح	المعنى المختصر
API	عقد تتواصل عبره الواجهة مع الخادم
Endpoint	عنوان عملية محددة داخل API
DTO	شكل بيانات للنقل لا يساوي كيان قاعدة البيانات
RBAC	صلاحيات حسب الدور
JWT/Token	إثبات هوية موقع يرسل مع الطلب
ORM / EF Core	تحويل الكيانات والاستعلامات إلى قاعدة البيانات
Migration	نسخة تغيير منظمة لمخطط قاعدة البيانات
Transaction	مجموعة عمليات تنجح كلها أو تتراجع كلها
Concurrency	حماية من تعديل سجل نفسه في الوقت ذاته
Microservice	خدمة مستقلة بمهمة وحدود واضحة
RAG	استرجاع معرفة ثم توليد جواب مدعوم بها
Embedding	تمثيل عددي لمعنى النص
RRF	دمج قوائم نتائج بحسب مراتبها
CrossEncoder	إعادة ترتيب دقيقة لسؤال مع مقطع
Structured Output	خرج JSON مفروض بمخطط
Redis	مخزن سريع للكاش والجلسات والحدود
OCR	استخراج نص من صورة
Haversine	مسافة هوائية تقريبية بين إحداثيات
OSRM	محرك مسارات طرق مفتوح
FEFO	الأقرب انتهاء يخرج أولاً
Fallback	مسار بديل عند فشل مكون

28. ورقة المراجعة قبل المناقشة

تأكد أنك تستطيع شرح الآتي من دون قراءة

- جملة واحدة: المشكلة والحل والقيمة.
- الأدوار الستة ولماذا صلاحياتها منفصلة.
- تدفق request من React/Flutter إلى ASP.NET ثم DB/AI.
- الفرق بين مطابقة اسم الدواء والتحقق من توفره.
- الفرق بين Haversine وOSRM ودور Google Places.
- دورة طلب التوريد والتخصيص من أكثر من دفعة.
- دورة التبرع ولماذا الصورة ليست قبولا نهائيا.
- خطوات Hybrid RAG ومعنى RRF وCrossEncoder والمصادر.
- معادلة توقع النفاذ ولماذا هي Decision Support حاليا.
- كيف تحمي الصلاحيات والمفاتيح والبيانات الحساسة.
- ثلاث نقاط قوة وثلاثة قيود وخطة تطوير واقعية.

آخر نصيحة

كذا، والجزء التالي يحتاج تحققاً/بيانات إضافية، وخطة تطويره هي كذا. هذه إجابة هندسية أفضل من ادعاء غير قابل للإثبات. إذا لم تعرف جواباً دقيقاً، لا تخمن. قل: في النسخة الحالية نفدنا

جملة ختامية مقترحة

الخلاصة

وتستخدم الذكاء كطبقة مساعدة قابلة للتفسير والضبط، مع إبقاء القرار الطبي والقانوني والمالي لدى أصحاب الصلاحية. دوائي توحد الوصول إلى الدواء مع التشغيل الصيدلاني وسلسلة التوريد والتبرعات،

29. من أين تبدأ قراءة الكود؟ خريطة قراءة الكود

خطة عملية تربط ما فهمته في الدليل بالملفات الفعلية.

أبدأ هنا	ماذا ستعرف؟
Front-End / src/app/routes	المسارات والصفحات والأدوار التي تستطيع فتحها
Front-End / src/features	كل مجال وظيفي: user, pharmacy, warehouse, admin...
Front-End / src/shared	المكونات والإعدادات والأدوار والعملاء المشتركة
Backend / Controllers	عقد الـ API والـ endpoints والسياسات
Backend / Application	Validators وواجهات الخدمات و DTOs
Backend / Infrastructure/Services	تنفيذ سيناريوهات العمل والتكاملات
Backend / Persistence/PharmacyDbContext	الجدول والعلاقات والقيود والفهارس
services_AI / docker-compose.yml	الخدمات والمنافذ و Redis و health checks
services_AI / smart_pharmacy_bot/app.py	والبدائل والتنبؤ والوصفة RAG
services_AI / drug_search_api_out	تطبيع ومطابقة أسماء الأدوية
services_AI / license_ai_service	والتحقق الأولي من الترخيص OCR
Flutter / lib	الشاشات والمستودعات البرمجية والربط مع API

طريقة تتبع ميزة واحدة

- 1 ابدأ من route أو الزر في الواجهة وحدد page/component.
- 2 ابحث عن hook أو api client الذي يرسل الطلب.
- 3 انتقل إلى Controller endpoint المطابق في الباك.
- 4 اتبع interface ثم service implementation ومنطق التحقق والصلاحيات.
- 5 راجع DbContext/Entity لمعرفة ما يخزن والعلاقات.
- 6 إن كان هناك AI، اتبع HTTP client إلى endpoint خدمة Python.
- 7 اختتم بالاختبارات والـ logs وحالات الخطأ، لا بالمسار الناجح فقط.

تمرين مقترح

← الشحنة ← التسليم ← الفاتورة. إذا استطعت شرحه من الواجهة إلى قاعدة البيانات فأنت فهمت قلب المشروع.
«طلب توريد من دفعتين» كاملاً: صفحة الصيدلية ← API الطلب ← كيان OrderItem ← قبول المستودع ← BatchAllocation
تتبع سيناريو